UNIT 3 · CO3

Pipelining — Student Notes

Computer Organization and Architecture (CSE23CT201) · Sri Ramachandra Faculty of Engineering and Technology

A standalone reading handout — bullet points, all figures, and questions under each topic. Follows Hamacher, Vranesic & Zaky, Chapter 6.

Contents

1. The Pipelining Idea
2. Pipeline Performance
3. Data Hazards
4. Instruction (Control) Hazards
5. Influence on Instruction Sets
6. Datapath & Control Considerations
7. Superscalar Operation

1. The Pipelining Idea

Hamacher §6.1 – §6.2

1.1 Why pipelining?

- A program can be made faster in **two** broad ways:
  - use **faster circuitry**, or
  - arrange the hardware so that **more than one operation happens at the same time**.
- Pipelining takes the **second** route — it **overlaps** the execution of successive instructions.
- **Analogy — the assembly line:** a car takes hours to build, yet a finished car rolls off the line every few minutes, because every station is always working on a different car.

1.2 One instruction = five steps (recall from Unit 2)

- A single instruction executes in five steps: **Fetch** → **Decode** → **Compute** → **Memory** → **Write** (5 clock cycles).
- Each step uses **different hardware**.
- So the moment instruction I_j leaves Fetch and enters Decode, the Fetch unit is **free** — it can fetch I_{j+1} right away.
- Overlapping the steps of successive instructions across dedicated hardware stages is **pipelining**.

1.3 The ideal case

- Three instructions are overlapped so that every stage is busy with a **different** instruction (see figure).
- Any one instruction still takes 5 cycles start-to-finish, **but** once the pipeline is **full**, a new instruction **completes every cycle**.

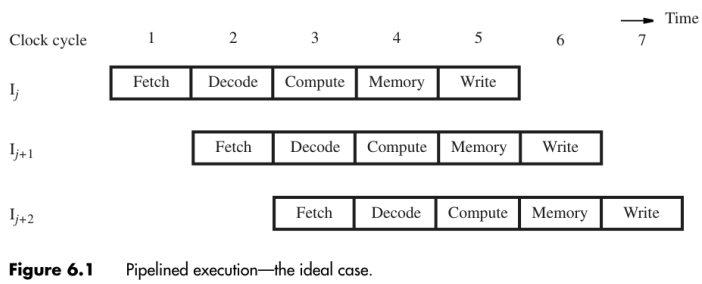


Figure 6.1 Pipelined execution—the ideal case.

Figure 6.1 — Pipelined execution, the ideal case. Once full, one instruction completes every cycle.

Key idea. Pipelining does **not** make one instruction faster. It increases **throughput** — instructions completed per unit time — by overlapping many instructions.

1.4 Pipeline organization & interstage buffers

- Each of the five stages is **separate hardware**.

- Between consecutive stages sit registers called **interstage buffers** (B1–B4).
- At the end of every clock cycle, each buffer **latches** its stage's result so the next stage can use it in the following cycle.
- The buffers carry data, register identifiers **and** the control signals for later stages.

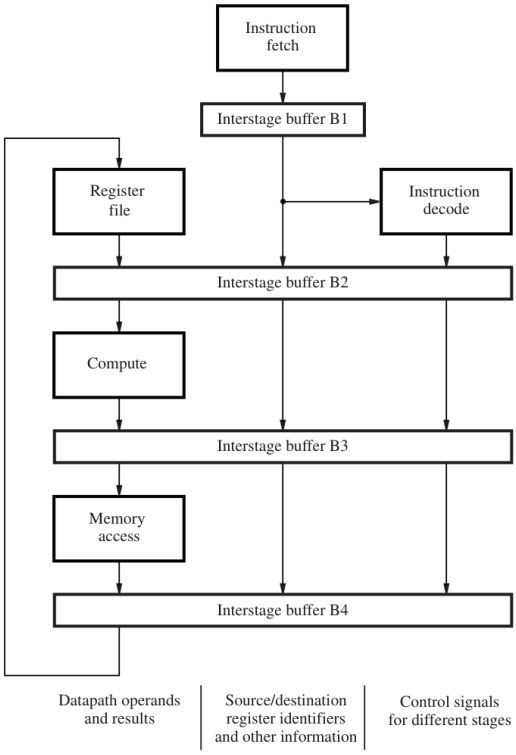


Figure 6.2 A five-stage pipeline.

Figure 6.2 — A five-stage pipeline (stage hardware + interstage buffers B1–B4).

- **B1** → the newly fetched instruction, ready to decode.
- **B2** → the two operands read from registers, the immediate value, and the **control signals** for later stages.
- **B3** → the ALU result (data to write, or an address/data for memory).
- **B4** → the value to be written back into the register file.
- Control signals travel **with** the instruction down the pipe — this is **pipelined control**.

QUESTIONS — Chapter 1

1. Define pipelining. **[1 mark · Remember]**
2. Name the five stages of the pipeline used in this chapter. **[1 mark · Remember]**
3. What is an interstage buffer? What does each of B1–B4 carry? **[2 marks · Understand]**
4. With a neat diagram (Figure 6.1), explain the ideal overlapped execution of three instructions. **[5 marks · Apply]**

---

2. Pipeline Performance

Hamacher §6.8 (basics)

2.1 Counting the cycles

- Take a pipeline of **k** stages executing **n** instructions.
- The first instruction needs k cycles to reach the end; after that, one completes each cycle.
- Total time = **k + (n − 1)** cycles — compared with **k × n** cycles with no overlap.
- Example: k = 5, n = 100 → **104** cycles instead of 500 (≈ **4.8× faster**).
- As n grows large, the speedup approaches **k** (the number of stages).

Worked example A 5-stage pipeline (k = 5) runs a loop with n = 20 independent instructions. Cycles needed = k + (n − 1) = 5 + 19 = **24 cycles**. Without pipelining it would take k × n = 5 × 20 = **100 cycles**. Speedup = 100 / 24 ≈ **4.17×** — close to, but below, the ideal 5× because n is still small;

as n grows the answer creeps closer to k.

→ Go deeper, with more worked examples, in the Foothold course — open **Module 3 → 3.2 The Pipeline Idea & Performance**
Optional — only if you want to go deeper. Not needed for the exam.

2.2 The performance equation

- For a program with dynamic instruction count N:

$$T = (N \times S) / R$$

N = number of instructions · S = average cycles per instruction · R = clock rate

- **Throughput** = instructions executed per second:

$$P_{np} = R / S \;\rightarrow\; \text{ideal pipeline } (S = 1)\text{: } P_p = R$$

- Non-pipelined: S = 5. Ideal pipelined (no stalls): S = 1 → up to a **5× throughput** gain.

2.3 How many stages?

- **More stages** → less work per stage → shorter clock period → **higher clock rate**.
- **But** more instructions in flight → more dependencies → more stalls.
- **And** the branch penalty grows if the branch decision moves to a later stage.
- The clock is limited by the **slowest basic operation** (usually the ALU).
- Gains **diminish** beyond a point — real designs use ~10–20 stages at several GHz.

Key idea. Ideal speedup approaches the number of stages; every stall pulls the real gain below the ideal — which is what the hazards (next) are about.

QUESTIONS — Chapter 2

1. Differentiate **throughput** from the execution time of a single instruction. **[2 marks · Understand]**
2. Derive the number of cycles for n instructions on a k-stage pipeline, and hence the speedup as n → ∞. **[2 marks · Apply]**
3. Write the basic performance equation T = N·S/R and explain each term; give S for the non-pipelined and ideal-pipelined cases. **[2 marks · Understand]**
4. Discuss the trade-offs in choosing the number of pipeline stages. **[5 marks · Analyse]**

---

3. Data Hazards

Hamacher §6.3 – §6.5

3.1 The data dependency

- The ideal overlap assumes instructions are **independent** — real code often chains them.
- Example: `Add R2,R3,#100` then `Subtract R9,R2,#30` .
  - R2 is **produced** by the Add and **consumed** by the Subtract — a **data dependency**.
  - The Add writes R2 in cycle 5 (Write), but the Subtract reads it in cycle 3 (Decode) — a **read-before-write**.
- Any condition that forces the pipeline to wait is a **hazard**; this one is a **data hazard**. This particular kind — a later instruction depending on an earlier one's result — is called a **RAW (Read-After-Write) hazard**.

EXPLANATION
*The name "Read After Write" can feel backwards right after reading "read-before-write" above — but the two phrases describe two different things. **RAW names the dependency**: in correct program order, the Write must happen, then the Read happens after it — that's simply what "R2 is produced by the Add and consumed by the Subtract" means. **The hazard is that pipelining breaks this order** — the Subtract's Decode tries to read R2 before the Add's Write has actually happened. So the dependency is named for the order that should hold (Write, then Read); the hazard is that the pipeline lets the Read jump ahead of it. The same logic gives WAR (Write-After-Read) and WAW (Write-After-Write) hazards elsewhere in the course.*

3.2 The simplest cure — stall

- **Stall** the Subtract in Decode until R2 is written — here for **3 cycles**.
- Each idle cycle inserted is a **bubble**; all following instructions are delayed too.

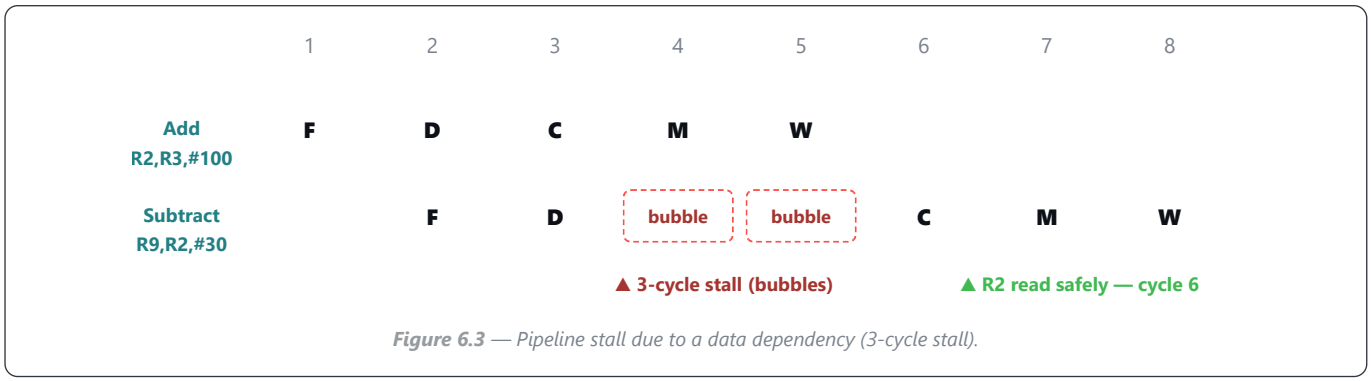


Figure 6.3 — Pipeline stall due to a data dependency (3-cycle stall).

EXPLANATION

*Look carefully at the diagram: the Subtract's **D** box (cycle 3) is followed by only **two** cells marked "bubble" (cycles 4 and 5) — yet we call this a **3-cycle stall**. Both numbers are correct; they're just counting different things. Cycle 3 was already going to be Subtract's normal Decode cycle — it isn't "extra" time. Cycles 4 and 5 are the two additional idle cycles forced in because R2 isn't ready yet — those are the two bubbles you see drawn. Add them together — cycle 3 (held) + cycle 4 (bubble) + cycle 5 (bubble) — and Subtract is stuck in Decode for a **total of 3 cycles** before it can move to Compute in cycle 6. So: 2 bubbles = the extra delay injected; 3 cycles = the full time spent stalled. If you check this same example on the AVPS simulator, its own numbers agree exactly — it also stalls for 3 cycles — but because the simulator advances one cycle at a time, you'll only ever see up to two bubble cards on screen together at once (the third stalled cycle's bubble hasn't reached the next stage yet). That's a feature of how the animation plays out, not a disagreement with the textbook.*

3.3 Operand forwarding — the better cure

- The value the Subtract needs actually **exists earlier** than the register write.
- The Add's ALU produces R2 at the **end of cycle 3** and it is captured in register **RZ**.
- **Operand forwarding**: route the result directly from RZ to the ALU input where it is needed — **no stall**.

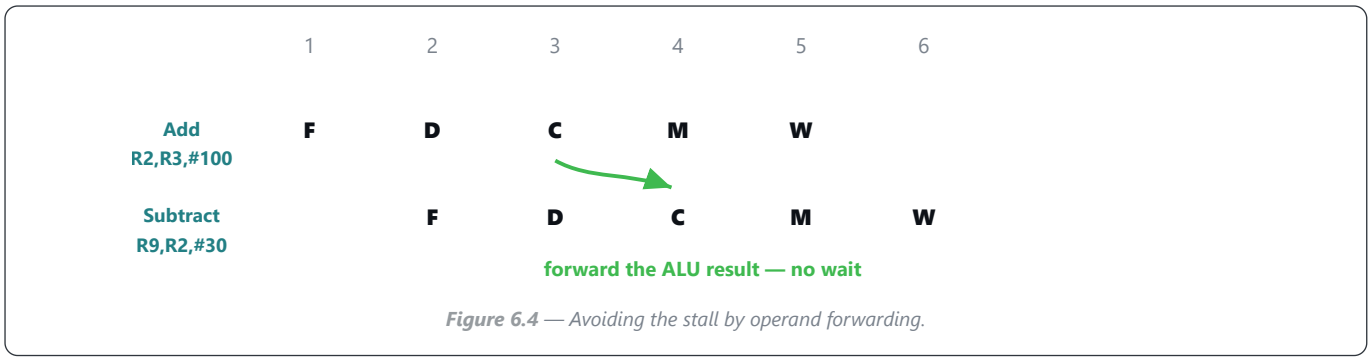


Figure 6.4 — Avoiding the stall by operand forwarding.

- The hardware needs a small change (see figure): a multiplexer (MuxA) before the ALU's InA input, and MuxB extended, so each input can select either a register value **or** the value forwarded from RZ.
- Forwarding can also be extended from register **RY** to handle a result produced two instructions earlier.

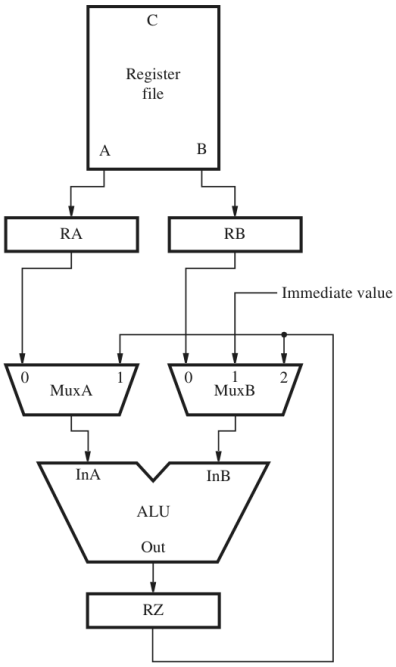


Figure 6.5 Modification of the datapath of Figure 5.8 to support data forwarding from register RZ to the ALU inputs.

Figure 6.5 — Datapath modified so RZ feeds back to the ALU inputs.

3.4 Worked example — forwarding two results (Example 6.1)

- Sequence: `Add R4,R3,R2` · `Or R7,R6,R5` · `Subtract R8,R7,R4` .
- The Subtract needs **both** R4 (from Add) and R7 (from Or).
- When the Subtract reaches Compute (cycle 5): R7 is forwarded from RZ, R4 is forwarded from RY → **no stall**.

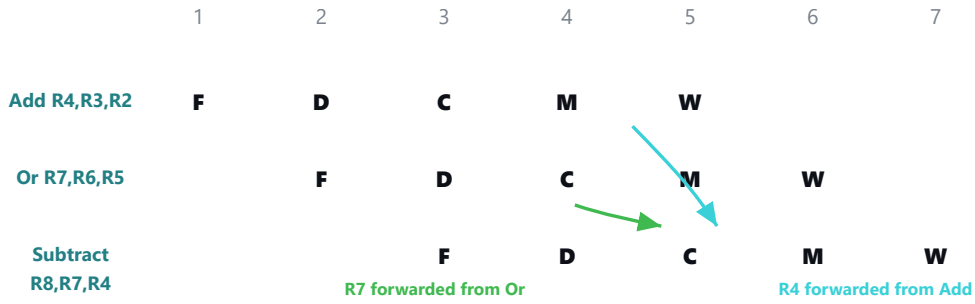


Figure 6.15 — Two operands forwarded into one instruction; no stall required.

3.5 Handling it in software

- Alternative to hardware forwarding: let the **compiler** fix it.
- **Insert 3 NOPs** between the dependent pair — creates the delay, but grows code and wastes cycles.
- **Better — reorder** independent instructions into those slots so the CPU does useful work: **instruction scheduling**.

```
Add     R2, R3, #100
NOP
NOP
NOP
Subtract R9, R2, #30
```

Three NOPs give R2 time to be written before the Subtract reads it.

Figure 6.6 *— Using NOP instructions (or reordering) to handle a dependency in software.*

3.6 The load-use hazard (a data hazard, not a memory delay)

- **This happens even when every memory access hits.** A **Load** produces its data in the **Memory** stage — one stage *later* than the ALU produces an arithmetic result (Compute).
- So an instruction that **uses the loaded register** and follows directly needs that value in *its* Compute stage — but it is not ready until the end of Memory, one cycle too late. Even with forwarding this costs **one** unavoidable stall; after that single bubble the value (now in RY) is forwarded on.
- Because it comes from the program's own data dependency, a **compiler can hide it** — slot an independent instruction between the Load and its user and the bubble disappears (this is the **instruction scheduling** of 3.5, applied to loads).

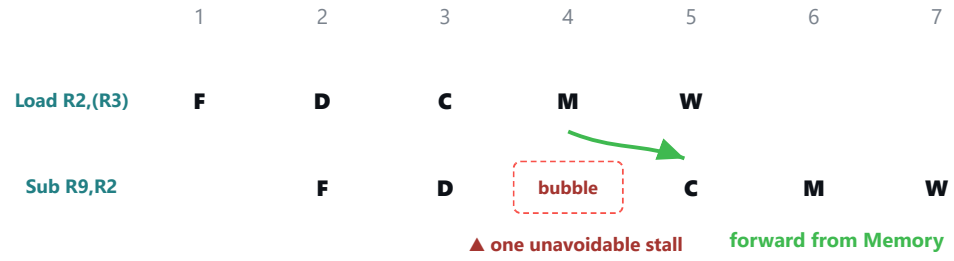


Figure 6.8 *— A load-use hazard: forwarding still needs a single-cycle stall.*

> **Don't confuse the two.** A **cache miss** (3.7) is a *memory-system* stall — variable, 10+ cycles, unpredictable, and it **cannot** be scheduled away. A **load-use hazard** (3.6) is a *data* hazard — a fixed **single** bubble that survives forwarding but that a compiler **can** remove. The word "Load" appears in both, which is why they get clubbed — but a single Load can suffer **both at once**, and the penalties add.

3.7 Memory delays (a memory-system stall)

- **The pipeline only flows if every stage finishes in one cycle.** The **Fetch** stage and the **Memory** stage both read the cache, which answers in about one cycle — but *only on a hit*.
- **Cache miss:** on a miss the cache must go down to main memory to fetch the block — **tens of cycles** (10+). The stage now has no valid word to pass on, so it **freezes**; and because the pipeline is strictly in-order, **every instruction behind it freezes too** (bubbles are injected) until the word arrives. That is why one slow access stalls *all following instructions*, not just the one that missed.
- **Two things to stress:** it is **non-deterministic** — the same Load stalls on a miss but not on a hit, so you cannot predict it by reading the code; and, unlike the load-use bubble in 3.6, you **cannot reschedule it away**.

> EXPLANATION
>
> ***Why can't we just reschedule a cache miss away?*** *The idea is tempting, because rescheduling is exactly how we kill the load-use bubble in 3.6: the compiler slides an independent instruction into the gap so the CPU does useful work instead of idling. To try the same trick here, the compiler would move some independent instructions to run after the Load, hoping they fill the memory wait. It fails for two reasons.* ***First, a miss is invisible at compile time*** *— the very same Load hits on one run and misses on the next depending on what happens to be in the cache, so the compiler never knows there will be a gap, or how big it is.* ***Second, even if it did know, the gap is huge and variable*** *(tens to hundreds of cycles), and there are almost never that many independent instructions lying nearby to fill it. Contrast the load-use case: exactly one cycle, every time — so a single instruction fills it perfectly. A cache miss is a run-time property of the memory system, so only the hardware can deal with it as it happens — prefetching, a non-blocking ("lockup-free") cache, or out-of-order execution that runs later independent instructions while the miss is being served. The compiler can hint (a software prefetch), but it cannot schedule the stall out of existence.*

Go deeper — how the hardware hides cache-miss latency → *Foothold course · prefetching, non-blocking caches & out-of-order execution*

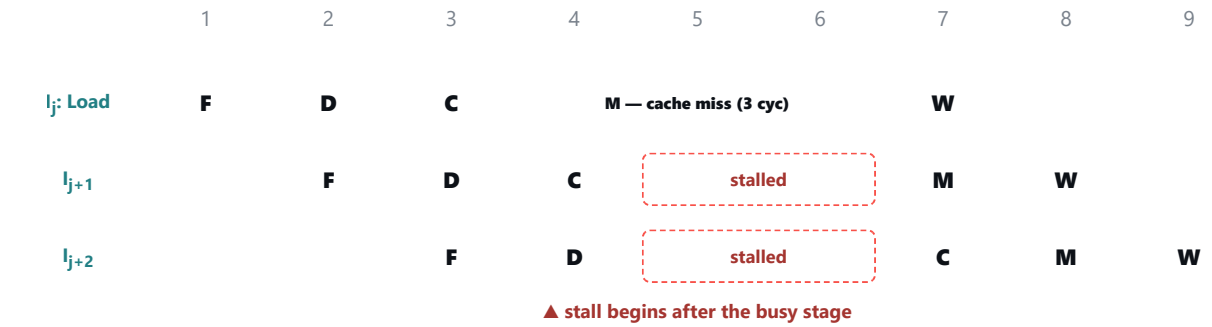


Figure 6.7 — Stall caused by a memory access delay for a Load instruction (drawn for a 3-cycle delay; a real cache miss costs 10+).

3.8 The Role of Cache Memory

- **Why the pipeline needs a cache.** A 5-stage pipeline wants to touch memory **every cycle** — the Fetch stage for the next instruction, and the Memory stage for a Load/Store. Main memory (DRAM) is roughly **100×** slower than the CPU needs, so if every access went straight to main memory the pipeline in Chapter 3 would stall constantly. A small, fast **cache** — built from SRAM and placed between the CPU and main memory — is what makes "one instruction per cycle" achievable at all.
- **The memory hierarchy:** registers (fastest, smallest) → L1 cache → L2/L3 cache → main memory (largest, slowest). Each step down trades speed for capacity; the cache levels exist purely to hide the gap between the CPU and DRAM.
- **Locality — why a small cache works at all:**
  - **Temporal locality**: a location referenced once is likely to be referenced again soon (e.g. a loop counter, a loop body's instructions).
  - **Spatial locality**: a location near one just referenced is likely to be referenced soon (e.g. the next instruction in sequence, the next array element).

  Real programs are predictable enough that a cache a tiny fraction of main memory's size still captures the large majority of accesses.

Hit, miss, and average access time Let **h** be the **hit ratio** (fraction of accesses the cache satisfies by itself), **thit** the cache's own access time, and **tmiss** the extra time added when the cache must fetch a block from main memory. The average access time seen by the pipeline is:

$$t_{avg} = h \times t_{hit} + (1 - h) \times (t_{hit} + t_{miss})$$

Example: thit = 1 cycle, tmiss = 20 cycles (the extra cost of going to main memory), h = 0.95. tavg = 0.95 × 1 + 0.05 × (1 + 20) = 0.95 + 1.05 = **2.0 cycles** — close to the ideal 1 cycle, even though a miss alone costs 21 cycles, because misses are rare.

EXPLANATION

This is exactly the "cache miss (3.7)" stall from the previous section, seen from the memory-system side instead of the pipeline side. 3.7 told you what happens to the pipeline when a Fetch or Load misses (every later instruction freezes for 10+ cycles). This section tells you why the miss is rare enough that the pipeline still works — because locality keeps the hit ratio high, so the huge miss penalty gets averaged down to only a little over 1 cycle per access.

- **Cache in the pipeline — a structural detail.** The Fetch stage and the Memory stage may both want to access **the same cycle** — if there is only one cache, one of them must wait. §6.2 in Chapter 6 covers this as a **structural hazard** and its fix: **separate instruction and data caches** (a Harvard split), so Fetch reads the I-cache while Memory reads the D-cache in the same cycle, with no conflict.

Key idea. A cache makes a slow main memory look fast to the pipeline — **locality** is why it works, and it is what lets the pipeline actually sustain close to **one instruction per cycle** in practice, not just in the ideal diagram of Chapter 1.

Go deeper, with the memory-hierarchy diagram and more worked examples, in the Foothold course — open **Module 3 → 3.1 The Role of Cache Memory**

Optional — only if you want to go deeper. Not needed for the exam.

QUESTIONS — Chapter 3

1. State what a "hazard" means in a pipeline; define a data hazard. **[1 mark · Remember]**
2. Explain operand forwarding in one or two sentences. **[2 marks · Understand]**
3. Why does a load-use dependency cost at least one stall even with forwarding? **[2 marks · Apply]**
4. For `Add R2,R3,#100` then `Subtract R9,R2,#30`, draw the pipeline timing **with** and **without** operand forwarding, and state the cycles saved. **[5 marks · Apply]**
5. How can a compiler handle a data dependency in software? Compare NOP insertion with instruction reordering. **[5 marks · Analyse]**
6. Why does a pipelined processor need a cache instead of accessing main memory directly? **[1 mark · Remember]**
7. Differentiate temporal locality from spatial locality, with one example of each. **[2 marks · Understand]**
8. A cache has hit time 1 cycle, miss penalty 20 cycles, and hit ratio 0.95. Compute the average memory access time, and comment on how close it is to the ideal. **[5 marks · Apply]**

---

4. Instruction (Control) Hazards

Hamacher §6.6

4.1 The branch problem

- To stay full, the pipeline fetches the **next in sequence** every cycle.
- A **branch** can redirect execution — but it must be **executed** before we know the outcome and target.
- The instructions already fetched behind it may be wrong and must be **discarded** — a **control (instruction) hazard**; the lost cycles are the **branch penalty**.

4.2 The branch penalty, and reducing it

- If the target is computed in the **Compute** stage: two instructions are fetched wrongly → **2-cycle** penalty.
- Branches are ~**20%** of instructions, so a 2-cycle penalty could add up to ~40% — worth attacking.

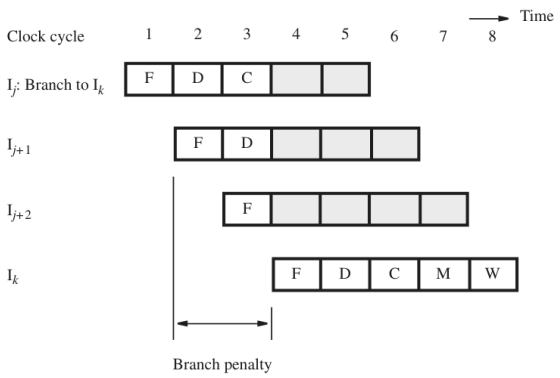


Figure 6.9 Branch penalty when the target address is determined in the Compute stage of the pipeline.

Figure 6.9 — Branch penalty when the target is determined in the Compute stage.

- **Decide earlier:** add an adder (and, for a conditional branch, a comparator) so the target and condition are worked out in the **Decode** stage. Only **one** instruction is fetched wrongly → **1-cycle** penalty for all branches.

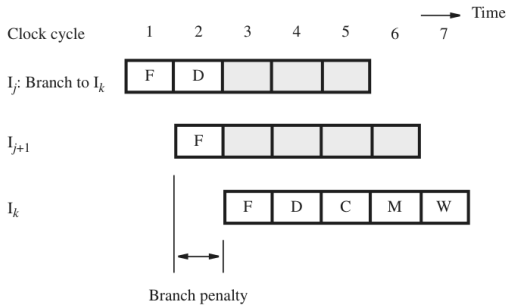


Figure 6.10 Branch penalty when the target address is determined in the Decode stage of the pipeline.

Figure 6.10 — Resolving the branch in Decode cuts the penalty from two cycles to one.

4.3 The branch delay slot

- The instruction location right after a branch is the **branch delay slot** — it is fetched anyway.
- **Idea:** always execute it, and let the compiler move a useful instruction there.
- Branching then takes effect one instruction later — **delayed branching**.
- Compilers fill the slot usefully about **70%** of the time (else a NOP is used).

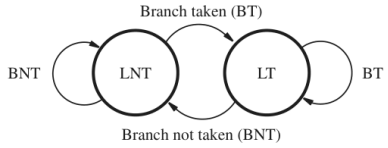
```
Add   R7, R8, R9                  Branch_if<R3=0> TARGET
Branch_if<R3=0> TARGET     ⟶     Add   R7, R8, R9   ← delay slot
NOP   ← wasted slot               ...
```

Before: a NOP wastes the slot. — *After: the Add moves into the slot — always executed.*

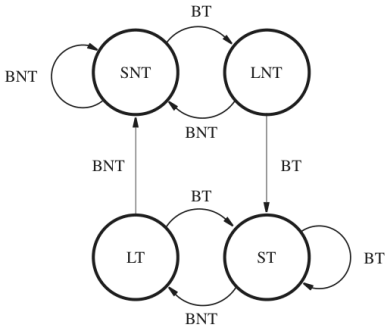
Figure 6.11 — Filling the branch delay slot with a useful instruction.

4.4 Branch prediction

- **Static prediction** — same guess every time:
  - **Assume not-taken** — keep fetching in sequence (~50% right for a random branch).
  - **Use direction** — a backward branch (loop end) is usually taken; predict by the sign of the offset.
  - A compiler-set **hint bit** can say "expect taken / not-taken".
- **Dynamic prediction** — learn from recent behaviour with a **2-bit** saturating counter (four states):
  - States: Strongly-Not-Taken, Likely-Not-Taken, Likely-Taken, Strongly-Taken.
  - Predict taken in the two "taken" states, not-taken in the two "not-taken" states.
  - It takes **two** wrong guesses in a row to flip — so one odd iteration won't fool it. For a loop run many times, only the exit pass is mispredicted.



(a) A 2-state algorithm



(b) A 4-state algorithm

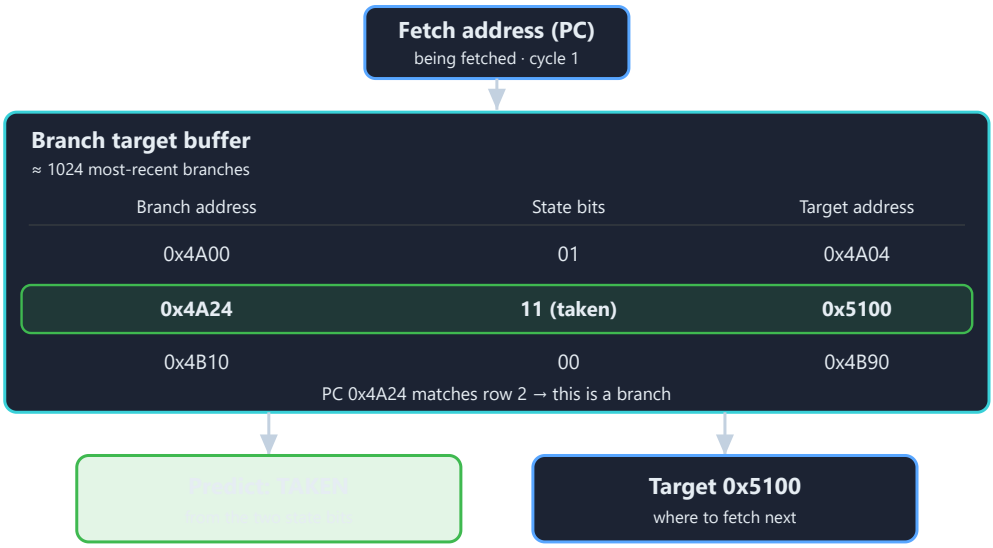
Figure 6.12 State-machine representation of branch prediction algorithms.

***Figure 6.12** — A 2-bit predictor: it flips its guess only after two misses in a row.*

Dynamic prediction in cycle 1 — the branch target buffer (BTB). We saw that the branch decision and the target can be worked out by the **Decode** stage (cycle 2). But the instruction fetched next to the branch in cycle 1 may be the wrong one — if so it is discarded and the correct instruction is fetched in cycle 3. To do better we must **predict in cycle 1**, at the same moment the branch itself is being fetched, and that means keeping some history of how the branch behaved before.

- That history is kept in a small, fast lookup memory — the **branch target buffer** — which identifies branches **by their address**. Each entry stores three fields: the **address of the branch instruction** (the tag to match on), **one or two state bits** for the prediction algorithm (the 2-bit counter above), and the **branch target address**.
- **On every fetch**, the processor checks the buffer for an entry whose address matches the instruction being fetched. A **hit** means "this is a branch we have executed before", so the state bits give the prediction (taken / not-taken) *and* the target address is read out — both in the **same cycle the branch is fetched**.
- In **cycle 2** the processor uses that prediction to fetch the next instruction, while also computing the *actual* decision and target to check the guess. If correct, execution continues with **no penalty**; if wrong, the just-fetched instruction is discarded, the correct one is fetched in **cycle 3**, and the state bits are updated.
- A **miss** simply means the address is not in the buffer — carry on fetching in sequence; the branch is added when it later resolves.
- Real programs have far too many branches to hold them all; a buffer big enough would be large and slow to search. So it is kept **small — typically about 1024 entries** — holding only the most-recently-executed branches, with old entries replaced as new branches run.

> **WHY IT MATTERS**
> *The pay-off is timing, not cleverness. Because the entry is found from the fetch address, the prediction bits and the target emerge at the very moment the branch is being fetched — a 2-bit predictor on its own only gives a direction, and only once the instruction has been decoded.*



Branch target buffer *— the branch's address, looked up while it is still being fetched (cycle 1), returns the prediction and the target together.*

Key idea. Control hazards shrink three ways: decide the branch **early**, fill the **delay slot** with useful work, and **predict** the branch (static, dynamic 2-bit, and the BTB for cycle-1 prediction).

QUESTIONS — Chapter 4

1. What is the branch delay slot? **[1 mark · Understand]**
2. Explain how a 2-bit branch predictor avoids being fooled by one unusual loop iteration. **[2 marks · Understand]**
3. With diagrams, explain the branch penalty when the branch is resolved in the Compute stage versus the Decode stage. **[5 marks · Apply]**
4. Compare the three techniques for reducing the branch penalty (early decision, delay slot, branch prediction), giving one advantage and one limitation of each. **[5 marks · Evaluate]**
5. What is a branch target buffer and why is it needed? **[2 marks · Understand]**

---

5. Influence on Instruction Sets

Hamacher §6.10

5.1 Pipelining rewards simple instructions

- The five-stage pipeline is **tailored to RISC-style** instructions; the wish to pipeline actually shaped instruction-set design.
- **RISC — pipeline-friendly:**
  - **Uniform length** (one word), few formats → simple, fast Fetch and Decode.
  - **Load–Store** architecture: only Load/Store touch memory; arithmetic is register-to-register → predictable timing.
  - Simple, regular **addressing** → simple operand-stage control.
- **CISC — pipeline friction:**
  - **Variable length** → several cycles just to fetch.
  - **Multiple memory operands** per instruction → extra stalls.
  - Complex addressing → **side effects** and condition codes.

5.2 Side effects and condition codes

- A **side effect** changes a location other than the named destination.
- Example: `Move R5, (R8)+` (autoincrement) writes R5 **and** also changes R8 — a hidden dependency that needs extra stall/forward logic.
- **Condition codes** are another side effect: a `Compare` sets flags that a following `Branch` silently depends on.
- Modern CPUs keep CISC instruction sets but internally **crack each into simple RISC-like micro-operations**.

Key idea. The instruction set and the pipeline are designed **together**. Simplicity and regularity — uniform length, load-store, simple addressing — are what let overlapping pay off.

QUESTIONS — Chapter 5

1. List three features of a RISC instruction set that make it easy to pipeline. **[2 marks · Understand]**
2. What is a "side effect"? Give an example and explain the problem it causes in a pipeline. **[2 marks · Apply]**
3. Explain how pipelining influenced instruction-set design, contrasting RISC and CISC. **[5 marks · Analyse]**

---

6. Datapath & Control Considerations

Hamacher §6.2, §6.7, §6.8.1

6.1 Per-stage hardware and pipelined control

- Each stage must have its **own hardware** — if two stages shared a unit, they couldn't both work in one cycle and the overlap would be lost.
- Each stage owns a **buffer register** (B1–B4) to hold its result for the next stage.
- **Pipelined control**: control signals are generated in Decode and travel **with** the instruction down the pipe.

6.2 Structural hazards

- A **structural hazard** occurs when two stages need the **same hardware resource** in the same cycle.
- Classic case: a single cache serves both **Fetch** (every cycle) and **Memory access** (loads/stores). Together they collide → one must stall.
- **Remedy:** provide **separate instruction and data caches** so Fetch and Memory proceed together.



§6.7 — Separate instruction & data caches remove the structural hazard.

6.3 Putting numbers on the stalls

- The ideal is S = 1. Each kind of stall adds a penalty term, and the terms are **additive**:

$$S = 1 + \delta_{\text{stall}} + \delta_{\text{branch}} + \delta_{\text{miss}}$$

- δ_{stall} (load-use): e.g. 0.25 × 0.40 × 1 = **0.10**.
- δ_{branch} (mispredict): e.g. 0.20 × 0.10 × 1 = **0.02**.
- δ_{miss} (cache — often the **biggest**): e.g. (0.05 + 0.30 × 0.10) × 10 = **0.8**.

EXAMPLE 6.2

- **Given:** 20% branches, static not-taken prediction (1-cycle penalty), no other stalls.
- 30% taken → δ_{branch} = 0.20 × 0.30 = 0.06 → **S = 1.06**.
- 70% taken → δ_{branch} = 0.20 × 0.70 = 0.14 → **S = 1.14**.
- Speedup of the faster case = (1.14 / 1.06 − 1) × 100 ≈ **7.5%**.

Key idea. Give every stage its own hardware and duplicate the shared resources that would clash; when you put numbers on the stalls, the **cache-miss** term usually dominates.

QUESTIONS — Chapter 6

1. Define a structural hazard and give one example. **[1 mark · Remember]**
2. How do separate instruction and data caches remove a structural hazard? **[2 marks · Understand]**
3. A program has 20% branches with a 1-cycle penalty and static not-taken prediction. Derive S and the % slowdown when 30% vs 70% of branches are taken (Example 6.2). **[5 marks · Analyse]**

---

7. Superscalar Operation

Hamacher §6.9

7.1 Multiple issue

- A single pipeline tops out at **one instruction per cycle**.
- A **superscalar** processor has **several execution units** and issues multiple instructions each cycle (**multiple-issue**).
- A **fetch unit** fetches 2+ instructions per cycle into an **instruction queue**; a **dispatch unit** sends independent instructions to different execution units.

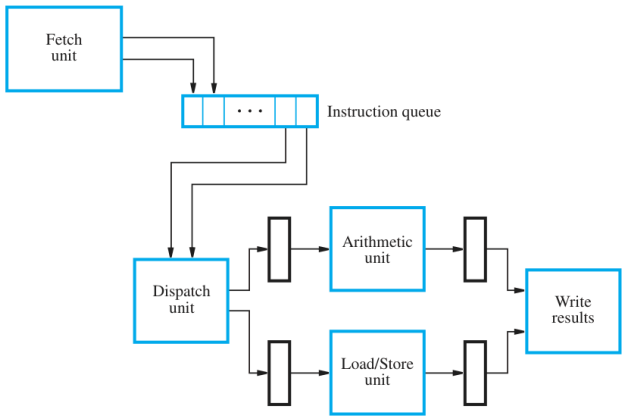


Figure 6.13 A superscalar processor with two execution units.

***Figure 6.13** — A superscalar processor with two execution units.*

- Here an arithmetic instruction and a Load/Store can be **issued together** each cycle, so a four-instruction sequence finishes far sooner.

	1	2	3	4	5	6
`Add R2,R3,#100`	**F**	**D**	**C**	**W**		
`Load R5,16(R6)`	**F**	**D**	**C**	**M**	**W**	
`Subtract R7,R8,R9`		**F**	**D**	**C**	**W**	
`Store R10,24(R11)`		**F**	**D**	**C**	**M**	**W**

***Figure 6.14** — Two instructions issued per cycle (arithmetic + Load/Store).*

7.2 Keeping many-in-flight correct

- With several units running, instructions may finish **out of order**; extra hardware keeps results correct and exceptions **precise**:
- **Reservation stations** — hold a dispatched instruction until all its operands arrive.
- **Out-of-order execution** — independent instructions run as soon as they are ready.
- **Register renaming** — temporary registers avoid false conflicts.
- **Reorder buffer** — commits results **in program order** → precise exceptions.

7.3 The three hazards — summary

- **Data hazard** — cause: an instruction needs a result not yet produced. Remedy: forwarding; stall; instruction scheduling.
- **Control (instruction) hazard** — cause: branch outcome/target not yet known. Remedy: decide early; delay slot; branch prediction + BTB.

- **Structural hazard** — cause: two stages need the same hardware resource. Remedy: duplicate resources (separate I-cache / D-cache).

Key idea. Pipelining approaches one instruction per cycle; superscalar operation pushes past it to **several per cycle**. Realising the potential depends on the instruction set, the pipeline hardware, and the compiler working together.

QUESTIONS — Chapter 7

1. What is a superscalar processor? How does it differ from a simple pipeline? **[2 marks · Understand]**
2. Name and explain any two mechanisms that keep out-of-order execution correct. **[2 marks · Understand]**
3. Draw the organization of a superscalar processor with two execution units and explain how two instructions are issued per cycle. **[5 marks · Apply]**
4. Tabulate the three hazard types with their cause and one remedy each. **[5 marks · Understand]**

Source: C. Hamacher, Z. Vranesic, S. Zaky, *Computer Organization and Embedded Systems*, 5th edition, McGraw-Hill, 2011 — Chapter 6 (Pipelining). Figures redrawn in colour from the textbook. Companion to the classroom deck.